

FOODPLAZA

Restaurant-Owned Food Delivery Platform

Production-Grade Project Documentation

A full-stack restaurant ordering and delivery management system

Customer • Admin • Delivery Boy

Project Definition

FoodPlaza is a restaurant-owned food delivery website where customers can browse the restaurant menu, manage a cart, place orders using Cash on Delivery or Razorpay, track delivery in real time, and interact with the restaurant through order cancellation requests, reviews, and feedback.

Document scope

This document consolidates the implementation details supplied in the original FoodPlaza project documentation.

1. Executive Overview

FoodPlaza is a full-stack, restaurant-owned food ordering and delivery platform built with Next.js on the frontend and an Express/Node.js backend connected to MongoDB Atlas. The platform is designed around three operational roles—Admin, User, and Delivery Boy—and supports the complete order journey from authentication and food discovery through checkout, restaurant processing, delivery assignment, OTP-based delivery confirmation, and post-delivery feedback.

A key focus of the implementation is real-time operational coordination. Socket.IO is used to propagate order, shop-status, and delivery events between the Admin, User, and Delivery Boy interfaces, while browser geolocation, Geoapify, and Leaflet/React-Leaflet support location-aware delivery workflows.

Implementation principle

The documentation below describes the features, libraries, workflows, and implementation approaches explicitly documented for FoodPlaza.

2. Product Scope & User Roles

Role	Primary responsibilities
Customer / User	Browse food, manage cart, select or save delivery addresses, place orders, track delivery, request cancellation, review food, and view/search order history.
Restaurant Admin	Control shop availability, manage food items, process orders, handle cancellation requests, inspect order history, and manage profile information.
Delivery Boy	View nearby available orders, accept/decline assignments, complete deliveries, manage profile information, and review daily/weekly/monthly earnings.

3. Core Feature Set

3.1 Authentication & Account Management

- Three account types: Admin, User, and Delivery Boy.
- Two login methods: email/password and Google authentication.
- Email/password authentication compares the supplied email with the database and verifies the password using bcrypt.
- Google authentication uses Firebase services and matches the authenticated email against the stored account.
- A single Admin account is permitted to be created.
- Delivery Boy signup requires a unique Aadhaar number.
- Forgot/reset password workflow uses three stages: send OTP to email, verify OTP, and set a new password.
- At login time, the current location of the User or Delivery Boy is stored.

3.2 Authorization Middleware

Middleware	Purpose
isAdminLogin	Checks whether an Admin is authenticated.
isUserLogin	Checks whether a User is authenticated.
isDeliveryboyLogin	Checks whether a Delivery Boy is authenticated.
isEitherUserOrDeliveryboy	Identifies whether the authenticated account is a User or Delivery Boy for shared location-related workflows.

3.3 Customer Experience

- View and update profile details.
- Browse individual food details, descriptions, related foods, and customer reviews.
- Use a custom image magnifier to inspect food images.
- Add, remove, and update cart quantities without a full page reload.
- Proceed from cart to checkout.
- Choose Cash on Delivery or Razorpay online payment.
- Save multiple delivery addresses.
- Select a delivery address using a real map.
- Search order history by food name.
- View order details and request cancellation from the Admin.
- Track the Delivery Boy route on a real-time map during delivery.
- Rate food and provide feedback after delivery.

3.4 Admin Operations

- Open or close the restaurant with a single control.
- View and update Admin profile information.
- Add, edit, and delete food items.
- Manage orders and update order status.
- Approve or reject customer cancellation requests.
- View order history including order details, delivery date/time, delivery address, payment method, and Delivery Boy information.
- Search order history by Order ID or customer name.
- Receive new-order notifications and counter updates in the Admin interface.

3.5 Delivery Boy Operations

- View available orders within a documented 5 km distance from the user/order location.
- Accept or decline available orders.
- View completed/delivered orders.
- View daily, weekly, and monthly earnings through charts.
- View and update profile details and profile image.
- Receive new order assignments in real time when an Admin marks an order as out-for-delivery.

4. Technology Stack

4.1 Frontend

Technology / Package	Documented usage
Next.js	Primary frontend framework; project created in Next.js.
Redux Toolkit	Application state management.
Redux Persist	Persists Redux state across page loads/navigation.
Chart.js	Delivery Boy daily, weekly, and monthly earnings charts.
Firebase	Google authentication.
GSAP	Interactive UI animations.
Leaflet	Interactive maps.
nanoid	Generates random IDs for food orders.
react-dom	React DOM rendering support.
React Hook Form	Form handling and validation.
React-Leaflet	React integration for Leaflet maps.
react-responsive	Responsive behavior/control.
React Toastify	Toast-based notification system.
Socket.IO Client	Real-time client-side communication.
Swiper	Carousel/slider UI.

4.2 Backend

Technology / Package	Documented usage
Node.js / Express	Backend runtime and HTTP API/server framework.
bcrypt	Password encryption/hash verification workflow.
Cloudinary	Cloud image storage; image URLs are used in the application.
cookie-parser	Cookie handling.
cors	Cross-origin request handling.
dotenv	Environment configuration.
jsonwebtoken	JWT-based authentication support.
Mongoose	MongoDB object modeling.
Multer	Multipart/file upload handling.
Nodemailer	Email delivery, including delivery OTP.
Nodemon	Development-time server restart utility.
Razorpay	Online payment integration.
Socket.IO	Real-time server-side communication.

4.3 Database & Styling

Area	Technology
Database	MongoDB Atlas
Styling	Tailwind CSS
Animation	GSAP
Carousel / Slider	Swiper.js
Responsive targets	Mobile, desktop, and laptop/large-screen layouts

5. High-Level System Architecture

The documented architecture separates the customer-facing Next.js application, backend services, persistence, external services, and real-time communication layer.

Layer	Responsibilities	Documented technologies
Presentation / Frontend	Customer, Admin, and Delivery Boy interfaces; responsive UI; forms; cart; dashboards.	Next.js, React, Tailwind CSS, GSAP, Swiper, React Hook Form
State Management	Persist application state across	Redux Toolkit, Redux Persist

	navigation/page loads.	
API / Backend	Authentication, order processing, food management, delivery workflows, profile operations.	Node.js, Express, JWT, bcrypt, Mongoose
Database	Store application data including account, food, order, assignment, location and OTP-related data.	MongoDB Atlas
Real-Time Layer	Instant order/status/shop/assignment updates.	Socket.IO + socket.io-client
External Services	Google authentication, images, maps/geocoding, online payment, email.	Firebase, Cloudinary, Geoapify, Leaflet, Razorpay, Nodemailer

6. Location & Mapping System

6.1 Location Acquisition

- `useAskLocation` asks the user to allow browser location services when first entering the website.
- `useUpdateLocation` continuously updates location when the browser reports a location change.
- `navigator.geolocation.watchPosition()` is used to track current User and Delivery Boy positions.
- The latest latitude and longitude are sent to the backend and stored in MongoDB.
- The current location is also stored at User/Delivery Boy login time.

6.2 Geoapify & Leaflet

- Geoapify is used to obtain location information such as latitude, longitude, name, state, and pincode.
- Leaflet is used to render real maps.
- React-Leaflet connects the map experience with the React application.

6.3 Real-Time Delivery Tracking

1. The browser tracks current location with `navigator.geolocation.watchPosition()`.
2. Updated longitude and latitude are sent to the backend and stored in MongoDB.
3. The system identifies the authenticated User or Delivery Boy through the shared authentication middleware.
4. The User can track the Delivery Boy on a real map during delivery.
5. The documented system supports real-time tracking from both User and Delivery Boy sides.

7. Delivery Assignment & Distance Logic

When an Admin updates an order to an out-for-delivery state, FoodPlaza performs a documented availability and assignment flow:

6. Search for Delivery Boys within a 5 km distance of the order/user location.
7. Identify Delivery Boys who are currently busy.
8. Create a Set of busy Delivery Boy IDs for O(1) membership checks.
9. Determine available Delivery Boy IDs from the candidate set.
10. Create a delivery assignment and broadcast it to available Delivery Boys.
11. Store the assignment ID in the order model.

For individual available orders, the system calculates distance between the Delivery Boy location and delivery address. If a saved address does not contain latitude/longitude, distance calculation is skipped and the interface shows that the route map is not available.

8. Order Lifecycle

Stage	Documented behavior
-------	---------------------

1. Discovery	Customer browses foods and opens individual food details.
2. Cart	Customer adds/removes items and changes quantities without full-page reloads; debouncing is used.
3. Checkout	Customer proceeds to checkout and selects delivery address/payment method.
4. Payment	Cash on Delivery or Razorpay online payment.
5. Admin receipt	New order appears instantly in Admin Order section through Socket.IO.
6. Processing	Admin updates order status and can process cancellation requests.
7. Delivery assignment	For out-for-delivery orders, available Delivery Boys within the documented range receive the assignment.
8. Acceptance	Delivery Boy accepts or declines the order; acceptance is reflected instantly in Admin/User interfaces.
9. Live tracking	User can track the Delivery Boy route on a real-time map.
10. Delivery confirmation	Delivery Boy initiates delivery completion and the customer verifies a six-digit OTP.
11. Completion	Completed delivery status is reflected across relevant interfaces.
12. Feedback	Customer can rate the food and submit feedback after delivery.

9. Cart & Food Detail Experience

9.1 Individual Food Page

- Food item is displayed by food ID.
- Description is shown for the selected food.
- Custom image magnifier provides zoomed food-image viewing, with lens-size control for mobile.
- A side cart can be updated without reloading the page.
- Debouncing is used in side-cart interactions.
- Customer reviews are shown for the individual food.
- Up to three related foods are shown and can be added to the cart.

9.2 Cart

- All cart foods are displayed together.
- Quantity can be increased or decreased without a full-page reload.
- Price and UI are updated dynamically using debounced operations.
- Items can be removed without reloading the whole page.
- A Proceed to Checkout action navigates to the next stage.

10. Payment & Delivery Verification

10.1 Payment

- Cash on Delivery is supported.
- Razorpay online payment is supported.

10.2 Delivery OTP

12. Delivery Boy clicks Mark as Delivered.
13. A six-digit OTP is generated.
14. The OTP is saved in the database.
15. The OTP is configured with a five-minute expiry.
16. Nodemailer sends the OTP to the customer's email address.
17. Customer enters and verifies the OTP.

18. The resend workflow is available with a documented 30-second timer.

- `Math.floor()` is documented for six-digit OTP generation.
- `Date.now()` is used with a five-minute expiry calculation.
- Loading spinners and disabled states are used around delivery, verification, and resend actions.
- React Hook Form is used for input/form validation.

11. Real-Time Communication with Socket.IO

Socket.IO is a central part of FoodPlaza's operational experience. The documented implementation stores socket IDs for Admin, User, and Delivery Boy and propagates important events without requiring manual page refreshes.

Audience	Real-time behavior
Admin	Instantly receives new orders; sees Delivery Boy acceptance details; sees completed delivery updates; receives shop open/closed state changes; receives order counter notifications.
User	Order status updates appear instantly; Delivery Boy acceptance updates the order UI and exposes tracking; completed delivery is reflected immediately with Delivery Boy information.
Delivery Boy	New order assignments arrive instantly when an Admin moves an order to out-for-delivery.

12. Shop Open / Closed Control

- Admin has a dedicated shop open/close toggle.
- When the shop is closed, the customer-facing home page can display a Shop Closed card in the carousel.
- Admin can inspect and control the current shop state from the Admin Panel.
- Admin logout or taking a break after opening the shop does not itself change the customer-facing shop state.
- Closing the shop affects the customer-facing interface.
- A customer can still browse/login/build a cart while the shop is closed, but checkout/order placement is blocked with a shop-closed notification.
- The documented logic also prevents order placement from the order-summary/payment stage when the shop is closed.

13. Order Cancellation Workflow

Step	User	Admin
Request	Submits cancellation request for an order.	Receives the request in the Admin Order section.
Approve	Order becomes cancelled.	Approves the cancellation request.
Reject	Order continues for processing.	Rejects the cancellation request.

14. Earnings & Delivery Analytics

- `Chart.js` is used for Delivery Boy earnings visualization.
- Today's Earnings uses a bar chart.
- Weekly Earnings uses a line chart covering the last seven days.
- Monthly Earnings uses a bar chart with different colors for monthly totals.
- A Today's Earnings card shows the current day's total.
- A Total Earnings card shows cumulative earnings.

15. Notification & UI State Management

- React Toastify is used for user-facing toast notifications.
- Admin receives a new-order counter even when viewing another Admin page.
- AdminPanel state is passed to AdminOrder to trigger the documented UI rerender behavior for new orders.
- AdminOrder then updates its UI and calls the documented order-fetch API.
- Delivery Boy receives a counter notification when new orders arrive.
- Socket.IO and application state are combined to keep role-specific interfaces synchronized.

16. Responsive & Interactive UI

- The documented responsive targets are mobile and desktop/laptop/large-screen layouts.
- Tailwind CSS is used for styling.
- GSAP is used for interactive UI animation.
- Swiper is used for carousel/slider experiences.
- react-responsive supports responsive behavior.
- The custom image magnifier includes mobile-specific lens-size control.
- Dynamic cart interactions update UI without full-page reloads.

17. Forms, Validation & Notifications

Concern	Implementation
Form handling	React Hook Form
Password verification	bcrypt
Authentication	JWT plus documented role-specific middleware
Google authentication	Firebase
Notifications	React Toastify and Socket.IO-driven application events
Email OTP	Nodemailer
Loading/interaction feedback	Spinners and disabled action buttons in documented delivery OTP flows

18. Image & Media Handling

- Cloudinary is used to store images in the cloud.
- The application uses image URLs as image sources.
- Multer is used for backend multipart/file-upload handling.
- The customer food detail experience includes a custom image magnifier.

19. Selected Implementation Details

Implementation	Purpose / documented rationale
Debouncing	Reduces repeated update operations during cart quantity and side-cart interactions.
Set of busy Delivery Boy IDs	Provides O(1) membership checks while determining available Delivery Boys.
watchPosition()	Continuously observes browser location changes.
Role-aware middleware	Allows shared workflows to identify Admin/User/Delivery Boy authentication state.
Socket IDs	Identify connected role-specific clients for real-time events.
OTP expiry	Limits the validity window of delivery verification codes.
Persisted Redux state	Keeps application state available across page loads/navigation.
Dynamic UI updates	Order/cart interfaces update without requiring a full-page refresh.

20. Production-Oriented Security & Reliability Considerations Documented

The supplied documentation explicitly identifies several implementation mechanisms that contribute to the application's operational and security model:

- bcrypt is used for password verification against stored encrypted/hash values.
- JWT is included in the backend technology stack for authentication.

- Role-specific middleware protects Admin, User, and Delivery Boy workflows.
- OTP verification adds a delivery-time confirmation step.
- OTP expiry limits the documented delivery verification window to five minutes.
- Shop-closed checks prevent order placement when the restaurant is not accepting orders.
- Socket-driven state synchronization reduces stale operational views between roles.

Scope note

The source documentation lists these mechanisms but does not provide enough detail to claim a complete security audit, formal threat model, rate limiting, CSRF strategy, encryption-at-rest design, or production deployment configuration.

21. Representative Project Workflow

A typical customer-to-delivery workflow documented by the project can be represented as:

- Customer enters FoodPlaza and is prompted to allow location access.
 - Customer authenticates with email/password or Google authentication.
 - Customer browses the restaurant menu and opens a food detail page.
 - Customer inspects the food image, reviews, and related foods.
 - Customer adds food to the cart and adjusts quantities.
 - Customer proceeds to checkout and selects a saved/map-based delivery address.
 - Customer selects Cash on Delivery or Razorpay.
 - The order is received instantly in the Admin Order section.
 - Admin processes the order and moves it toward delivery.
 - When out-for-delivery is set, eligible available Delivery Boys are identified within the documented distance range.
 - Available Delivery Boys receive the assignment and can accept or decline it.
 - Acceptance is reflected instantly for Admin and User.
 - Customer can track the Delivery Boy on the real-time map.
 - Delivery Boy marks the order as delivered and triggers OTP verification.
 - Customer verifies the emailed six-digit OTP.
 - The order is completed and reflected across relevant interfaces.
 - Customer can rate the food and submit feedback.
- Dynamic cart and order state management
 - Delivery operations and earnings visualization

22. Feature Matrix

Feature	User	Admin	Delivery Boy
Authentication	✓	✓	✓
Google Authentication	✓	—	✓
Profile View / Update	✓	✓	✓
Food Management	Browse	Add / Edit / Delete	—
Cart	✓	—	—
Checkout	✓	—	—
Cash on Delivery	✓	Process	Deliver
Razorpay	✓	Process	—
Order Management	View / Request Cancel	Manage	Assigned Orders
Live Tracking	Track	Operational visibility	Share location
Order Cancellation	Request	Approve / Reject	—
Food Reviews	Rate / Feedback	View as applicable	—
Delivery Assignment	—	Trigger	Accept / Decline
Delivery OTP	Verify	Operational flow	Generate / Initiate
Earnings	—	—	Daily / Weekly / Monthly
Shop Open / Close	View effect	Control	—

23. Technology Summary

Category	Technologies
Frontend	Next.js, React, Redux Toolkit, Redux Persist
UI / Styling	Tailwind CSS, GSAP, Swiper, react-responsive
Forms / Notifications	React Hook Form, React Toastify
Maps / Location	Leaflet, React-Leaflet, Geoapify, browser Geolocation
Realtime	Socket.IO, socket.io-client
Authentication	Firebase Google Auth, JWT, bcrypt
Backend	Node.js, Express
Database	MongoDB Atlas, Mongoose
Files / Images	Multer, Cloudinary
Payments	Razorpay
Email	Nodemailer
Development utility	Nodemon

24. Conclusion

FoodPlaza is documented as a complete restaurant-owned food delivery ecosystem rather than only a customer ordering interface. Its implementation covers the customer ordering journey, restaurant administration, delivery operations, real-time communication, location-aware delivery workflows, online payment, delivery verification, and post-delivery feedback.

The strongest technical characteristics documented in the source are the three-role operational model, Socket.IO-based synchronization, browser-based live location tracking, distance-aware delivery assignment, OTP-based delivery completion, dynamic cart behavior with debouncing, persistent Redux state, and delivery earnings analytics.

25. Source Coverage & Documentation Note

This documentation was produced from the supplied 10-page FoodPlaza project document. Where the source provided implementation details, those details were retained and reorganized into a professional technical structure. Where the source did not provide enough information—for example, exact API route inventory, complete database schema, deployment infrastructure, CI/CD, automated tests, or detailed security controls—those areas were not presented as implemented facts.